

[Resources](#) > [Project / Exam](#) > [Example Programs](#) > StateMachonk



Date

Sort by



Marcus Peter Brendel Elmkvist

Xtext Syntax:

grammar org.xtext.example.statedsl.StateDSL **with** org.eclipse.xtext.common.Terminals

generate stateDSL "http://www.xtext.org/example/statedsl/StateDSL"

Domainmodel:

elements+=(StateMachine)*;

StateMachine:

```
'StateMachine' name=ID '{'  
'states' '=' '[' states+=(AState)+ ']' '  
'transitions' '=' '[' transitions+=(Transition)* ']' '  
'start' '=' start=State  
'};
```

AState:

State | StateMachine
;

State:

```
'State' name=ID '{' '}'  
  
;
```

Condition:

'Condition' name=ID ;

Transition:

'Transition' name=ID '{'

'from' '=' from=AState

'to' '=' to=AState

'condition' '=' condition=Condition

};

Example 1:

```

1 StateMachine state{
2     states = [
3         State state_idle,
4         State state_walk,
5         State state_jump
6     ],
7     transitions = [
8         Transition transition_idle {
9             from = walk{},
10            to = idle{},
11            condition = Condition idle_condition
12        },
13        Transition transition_walk {
14            from = idle{},
15            to = walk{},
16            condition = Condition walk_condition
17        },
18        Transition transition_walk {
19            from = idle{},
20            to = jump{},
21            condition = Condition jump_condition
22        },
23    ],
24    start = state_idle{},
25 }

```

Example 2:

StateMachine Base {

states = [

StateMachine Guard {

states = [

State idle {

}

```
State patrol {  
  
}  
  
State chase {  
  
}  
  
],  
  
transitions = [  
Transition idleToPatrol {  
from = State idle {}  
to = State patrol {}  
condition = Condition random  
}  
Transition patrolToIdle {  
from = State patrol {}  
to = State idle {}  
condition = Condition random  
}  
Transition idleToChase {  
from = State idle {}  
to = State chase {}  
condition = Condition enemyInSight  
}  
Transition patrolToChase {  
from = State patrol {}  
to = State chase {}  
condition = Condition enemyInSight  
}  
Transition ChaseToIdle {  
from = State chase {}  
to = State Idle {}  
condition = Condition enemyOutOfSight  
}  
],  
start = State Idle {}  
}
```

```
],  
  
transitions = [],  
start = State Guard{}  
}
```

Description of the output of a code generator:

We expect that we will get:

An abstract parent class called AState, from which both StateMachine and State classes inherit, ensures that the StateMachine can be treated as both a machine and a state. This abstract class should have three abstract methods: Enter(), Maintain(), and Exit() for running the states. It should also have a string variable called name, with getter and setter methods.

The StateMachine class contains a HashMap of states, with the state name as the key and the corresponding state object as the value. It also includes a HashMap of transitions, where the key is the state name and the value is an array of Transition objects, along with a starting state. Additional variables include currentState and nextState.

In the implementation of the Enter() method, the currentState is set to the startState, and the Enter() method of the currentState is called.

In the implementation of Maintain(), it calls the Maintain() method of the currentState and iterates through the Transition array in the transitions HashMap of the state. If the condition of a transition returns true, it calls the Exit() method on the current state, sets currentState equal to the transition target, and calls Enter() on this state object.

The State class includes a name and implements the Enter(), Maintain(), and Exit() methods from its parent.

The Condition class has a method that returns true if its criteria are fulfilled.

The Transition class accepts inputs: "from" of type AState, "to" of type AState, and "condition" of type Condition. This will be used to check each active state to determine if it is time to move to a specific state when the condition has been fulfilled.

posted 11-03-2026 14:23